IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2020

EEE/EIE PART II: MEng, BEng and ACGI

ALGORITHMS AND COMPLEXITY

Tuesday, 26 May 10:00 am

Time allowed: 1:30 hours

There are TWO questions on this paper.

Answer ALL questions. Question One carries 40% of the marks. Question Two carries 60%.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : D.F.M. Goodman
Second Marker(s) : D.B. Thomas

© Imperial College London

ALGORITHMS AND COMPLEXITY

1. a) For each of the following statements, state whether it is true or false and provide a supporting proof.

   i) $n(n+1)(n+2) = O(n^4)$. [3]

   ii) For every function $f(n) > 0$ we have $f(n) = \Theta(f(n))$. [3]

   iii) $e^{3n} = O(e^{2n})$. [3]

   iv) $6^n/(2^n+3^n) = \Theta(2^n)$. [3]

   b) Give a tight bound for each of the following recurrence relations, or explain why it is not possible to do so.

   i) $T(n) = \sqrt{2}T(n/2) + \sqrt{n}$. [2]

   ii) $T(n) = 3T(n/2) + n^3$. [2]

   iii) $T(n) = 4T(n/2) + \pi(n)$, where $\pi(n)$ is the number of prime numbers less than n. [4]

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) \;+\; O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

2. The *closest pair problem* is as follows. You are given a set of points in a plane $\mathbf{z}_i = (x_i, y_i)$ where $i = 1$ to n. The distance between two points with indices i and j is given by

$$d_{ij} = \|\mathbf{z}_i - \mathbf{z}_j\| = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

You need to find the distance between the closest pair of points,

$$d_{\min} = \min_{i \neq j} d_{ij}.$$

a) The following pseudocode is a brute-force algorithm to solve this problem.

```
1:  function ClosestPairBruteForce({z_i}, n)
2:      d_min = ∞
3:      for i = 1 to n do
4:          for j = 1 to n do
5:              if i ≠ j then
6:                  d = ||z_i − z_j||
7:                  if d < d_min then
8:                      d_min = d
9:                  end if
10:             end if
11:         end for
12:     end for
13:     return d_min
14: end function
```

i) Briefly summarise the operation of the algorithm in one or two English sentences. [2]

ii) What is the time complexity of the brute-force algorithm as a function of n? Justify your result. [3]

iii) Suggest a modification of the algorithm, changing only one line to make it run almost twice as fast. [3]

iv) Has this improved the time complexity of the improved version of the algorithm? [2]

b) Consider a divide and conquer approach to this problem, where we split the points up using their x-coordinates. This forms two equal subproblems, consisting of a "left" set and a "right" set of points.

Shown below is an incomplete pseudocode implementation of this algorithm, with missing lines indicated by "...".

The function MEDIAN($\{v_i\}$) returns the median of a set of values $\{v_i\}$, that is a number m such that half of the v_i satisfy $v_i \leq m$ and half satisfy $v_i > m$. The symbol x_i refers to the x-coordinate of the point $\mathbf{z}_i$.

```
1:  function CLOSESTPAIR({z_i}, n)
2:      if n ≤ 3 then
3:          return CLOSESTPAIRBRUTEFORCE({z_i}, n)
4:      end if
5:      m =MEDIAN({x_i for i = 1,...,n})
6:      L = {i for i = 1,...,n where x_i ≤ m}
7:      R = {j for j = 1,...,n where x_j > m}
8:      d^L_min =CLOSESTPAIR({z_i for i ∈ L}, LEN(L))
9:      d^R_min =CLOSESTPAIR({z_j for j ∈ R}, LEN(R))
10:     ...
11:     return d_min
12: end function
```

i) Briefly describe, and identify line numbers, for each of the following parts of the algorithm: (1) base case, (2) divide, (3) conquer. Note: the combine step is missing (line 10). [4]

ii) We can complete the combine step using the following logic.

Let (i^*, j^*) be the indices of the closest pair of points. There are three possibilities: (1) both points i^* and j^* are in the set L, (2) both points are in the set R, or (3) one point is in L and one point is in R. If the closest pair matches case (1) or (2), then it will be found in one of the two subproblems. So, for the missing combine step, only case (3) needs to be checked.

Write pseudocode for the missing combine step that checks distances between all pairs (i, j) where $i \in L$ and $j \in R$, and finds the distance between the closest pair in the whole set. Inserting this pseudocode at line 10 should complete the partial algorithm above. [4]

iii) What is the time complexity of your code for the combine step? Justify your result. [2]

iv) What is the complexity of this divide and conquer algorithm? Justify your result. You may assume that the median function is $O(n)$. [2]

c) The time complexity of this algorithm can be improved by modifying the combine step.

i) If the combine step could be reduced to a time complexity of $O(n)$, what would be the time complexity of the divide and conquer algorithm? [2]

ii) The combine step from (b)(ii) checks the distance between pairs of points with one point on the left and one on the right. Given $w > 0$, consider a pair of points (i, j) with $i \in L$ and $j \in R$ with $x_i < m - w$ or $x_j > m + w$.

What is the smallest possible value for $\|\mathbf{z}_i - \mathbf{z}_j\|$?

Suggest a modification of the combine step to avoid checking distances for points that must be further apart than the closest pair found in the divide and conquer steps.

Compute the time complexity for your modified combine step. You do not need to write pseudocode. [3]

iii) By applying the same reasoning to the y-coordinate, suggest a further improvement to the combine step that reduces the time complexity of the combine step to $O(n)$.

You may assume that all points have been pre-sorted with increasing y-coordinate.

Hint: use the fact that within any square of side length $d_{\min}$ there can be at most 4 points in L or 4 points in R. [3]